DORA and SPACE in the AI Age

Measuring What Matters When AI Changes Everything

How traditional software delivery and developer productivity frameworks must adapt for AI-assisted development. Introducing AI-DORA and AI-SPACE: modified metrics for the era of AI product builders.

6 Levels v1.1 Last updated June 2026

dora space ai-native metrics productivity devops

The DORA metrics and SPACE framework have been the gold standards for measuring software delivery performance and developer productivity. DORA's four key metrics—Deployment Frequency, Lead Time, Change Failure Rate, and Mean Time to Recovery—have guided engineering organizations for years. SPACE's five dimensions—Satisfaction, Performance, Activity, Communication, and Efficiency—provided a more holistic view of developer productivity.

But both frameworks were designed for a world where humans wrote code.

When AI can generate code faster than humans can type, when deployment frequency can increase 10x overnight, when the bottleneck shifts from implementation to validation—the traditional metrics don't just need adjustment. They need reimagining.

This article introduces **AI-DORA** and **AI-SPACE**: modified versions of these industry-standard frameworks for the era of AI-assisted development. We examine what changes, what stays the same, and how teams at different maturity levels should apply these frameworks.

The Original Frameworks

Before examining the modifications, we need to understand what DORA and SPACE measure and why they matter.

DORA: Software Delivery Performance

DORA (DevOps Research and Assessment) emerged from research by Nicole Forsgren, Jez Humble, and Gene Kim, published in *Accelerate* (2018). The research identified four metrics that predict software delivery performance:

Metric	What It Measures	Elite Threshold
Deployment Frequency	How often code reaches production	Multiple per day
Lead Time for Changes	Time from commit to production	Less than 1 day
Change Failure Rate	Percentage of deployments causing failures	0-5%
Mean Time to Recovery	Time to restore service after failure	Less than 1 hour

These metrics work together. High deployment frequency with high failure rate indicates recklessness. Low lead time with slow recovery suggests fragile systems. The four metrics form a balanced scorecard for delivery capability.

DORA's power is its empirical foundation. The research demonstrated that high-performing teams outperform low performers across all four metrics—there is no tradeoff between speed and stability for elite performers.

SPACE: Developer Productivity

SPACE was published in 2021 by researchers at Microsoft and GitHub (Forsgren, Storey, Maddila, Zimmermann, Houck, and Butler). It addresses a harder problem than DORA: measuring developer productivity without falling into the trap of counting lines of code or commits.

SPACE proposes five dimensions, and comprehensive implementations like LinearB's identify 20 individual metrics across them:

Dimension	Focus	Standard Metrics
Satisfaction	Developer well-being	Satisfaction surveys, eNPS, work-life balance, career growth
Performance	Quality of outputs	Code quality, feature delivery rate, velocity, defect density
Activity	Counts of actions	Commit frequency, PR activity, time in development
Communication	Collaboration effectiveness	Pair programming, cross-team collaboration, feedback loops, documentation
Efficiency	Flow state and focus	Cycle time, lead time, resource utilization, deployment frequency

SPACE's insight is that no single metric captures productivity. Different dimensions capture different aspects, and organizations should measure multiple dimensions to avoid Goodhart's Law—when a measure becomes a target, it ceases to be a good measure.

GitHub's Engineering System Metrics

GitHub's Well-Architected framework (2025) organizes 12 metrics into four zones, explicitly including AI-specific measurements:

Zone	Metrics
Quality	Change failure rate, recovery time, code security/maintainability
Velocity	Lead time, deployment frequency, PRs merged per developer
Developer Happiness	Flow state experience, tooling satisfaction, **Copilot satisfaction**
Business Outcomes	**AI leverage (%)**, engineering expenses to revenue, feature engineering %

The inclusion of **AI Leverage** and **Copilot Satisfaction** signals that mainstream frameworks now recognize AI as a first-class concern, not an afterthought.

Why AI Changes Everything

Both frameworks assume certain baseline realities that AI development disrupts:

Implementation is the bottleneck. Traditional metrics assume that writing code takes meaningful time. When AI generates code in seconds, the bottleneck shifts to specification, validation, and operations.

Human throughput scales linearly. A developer can review a certain number of PRs per day. With AI generating PRs at 10x the rate, review capacity—not generation capacity—becomes the constraint.

Lines of code correlate with value. SPACE explicitly warns against using LOC as a metric, but the Activity dimension still measures outputs. When AI can produce infinite outputs, counting them becomes meaningless.

Quality comes from human judgment. Both frameworks assume humans catch quality issues through code review. Level 6 development (as described in our **Software Delivery Autonomy** framework) eliminates human code review entirely.

Efficiency means minimizing interruptions. SPACE measures flow state in terms of uninterrupted human work time. With AI agents executing overnight, "flow" takes on a different meaning.

These are not minor calibration issues. They require rethinking what the metrics mean and how to measure them.

Introducing AI-DORA

AI-DORA maintains DORA's four core metrics but adjusts thresholds and adds context for AI-assisted development.

Deployment Frequency (AI-Modified)

Level	Standard DORA	AI-DORA
Elite	Multiple per day	10+ per day
High	Daily to weekly	Multiple per day
Medium	Weekly to monthly	Daily
Low	Monthly+	Weekly

What changes: AI-generated code enables dramatically higher deployment frequency. What was “elite” under standard DORA becomes merely “high” with AI assistance. Teams at Level 5+ (agentic engineering) routinely deploy 10 or more times daily.

What stays the same: The principle that elite teams deploy more frequently. Batch deployments remain riskier than small, frequent deployments.

AI enablers:

- AI code generation
- Automated test generation
- Intelligent CI/CD pipelines
- AI-assisted deployment decisions

Lead Time for Changes (AI-Modified)

Level	Standard DORA	AI-DORA
Elite	Less than 1 day	Less than 2 hours
High	1 day to 1 week	Less than 1 day
Medium	1 week to 1 month	1-3 days
Low	1 month+	1 week

What changes: AI collapses implementation time from hours or days to minutes. The constraint shifts to validation and deployment approval. Elite AI-assisted teams achieve sub-hour lead times.

What stays the same: Shorter lead times remain better. Fast feedback loops enable faster learning.

AI enablers:

- AI pair programming
- AI-assisted code review
- AI test generation
- Automated documentation

Change Failure Rate (AI-Modified)

Level	Standard DORA	AI-DORA
Elite	0-5%	0-2%
High	5-10%	2-5%
Medium	10-15%	5-8%
Low	15%+	8-12%

What changes: AI-generated code introduces new failure modes (hallucinations, subtle logic errors), but also enables more comprehensive testing. With proper validation infrastructure, AI-assisted teams should achieve lower failure rates than human-only teams.

What stays the same: Lower failure rates are better. High deployment frequency with high failure rate is worse than low deployment frequency with low failure rate.

AI enablers:

- AI static analysis
- AI-generated test coverage
- AI security scanning
- AI code review
- Scenario-based validation

Mean Time to Recovery (AI-Modified)

Level	Standard DORA	AI-DORA
Elite	Less than 1 hour	Less than 15 minutes
High	Less than 1 day	Less than 2 hours
Medium	1 day to 1 week	Less than 1 day
Low	1 week+	1-3 days

What changes: AI debugging assistants and root cause analysis dramatically accelerate recovery. AI can propose fixes, generate patches, and validate solutions faster than human troubleshooting.

What stays the same: Faster recovery is better. Recovery time often matters more than preventing all failures.

AI enablers:

- AI debugging assistants
- AI root cause analysis
- AI-powered runbooks
- AI fix suggestions
- Auto-remediation

Introducing AI-SPACE

AI-SPACE maintains SPACE's five dimensions but fundamentally reimagines what each measures in an AI-assisted context. The framework includes **25 metrics** (5 per dimension), drawing from the original SPACE research, GitHub's Well-Architected framework, and LinearB's comprehensive implementation.

S — Satisfaction and Well-being (5 metrics)

Aspect	Standard SPACE	AI-SPACE
Focus	Developer happiness	Human-AI collaboration satisfaction
Key Metric	Satisfaction survey	AI tool satisfaction + tooling satisfaction
New Consideration	N/A	Toil reduction, creative work ratio, work-life balance

AI-SPACE Metrics:

Metric	Description	Unit	Elite
AI Tool Satisfaction	Satisfaction with AI assistants (Copilot, Claude)	1-5	> 4.5
Engineering Tooling Satisfaction	Satisfaction with overall tool ecosystem	1-5	> 4.5
Toil Reduction	% reduction in repetitive work due to AI	%	> 70%
Creative Work Ratio	Time on creative vs routine tasks	%	> 80%
AI Work-Life Balance Impact	Whether AI improves or degrades balance	1-5	> 4.5

Why it matters: Satisfaction in AI development is not just "am I happy at work?" but "is AI helping or frustrating me?" Teams with poor AI tooling report lower satisfaction despite having "AI assistance." The work-life balance metric captures whether AI

enables sustainable work or creates new pressures (e.g., expectation of 24/7 availability because “AI can work overnight”).

P — Performance (5 metrics)

Aspect	Standard SPACE	AI-SPACE
Focus	Quality of outputs	AI-assisted output quality
Key Metric	Code quality, bugs	AI acceptance rate + defect density
New Consideration	N/A	Human oversight effectiveness, code security

AI-SPACE Metrics:

Metric	Description	Unit	Elite
AI Code Acceptance Rate	% of AI suggestions accepted after review	%	> 80%
AI Code Quality Score	Quality rating of AI-generated code	1-5	> 4.5
Human Review Catch Rate	Issues found in human review of AI code	%	5-15%
AI Code Defect Density	Defects per KLOC in AI-generated code	defects/KLOC	< 1
Code Security and Maintainability	Security/maintainability of AI code	1-5	> 4.5

Why it matters: When AI generates code, performance measurement must distinguish between AI quality and human review quality. A high acceptance rate could indicate good AI or rubber-stamping. The catch rate provides a counterbalance—too low suggests inadequate review, too high suggests poor AI quality.

The **5-15% optimal catch rate** is notable: Elite teams expect to catch some issues (demonstrating diligent review) but not many (indicating good AI quality).

A — Activity (5 metrics)

Aspect	Standard SPACE	AI-SPACE
Focus	Count of outputs	AI leverage and orchestration quality
Key Metric	PRs, commits, LOC	AI Leverage %, prompt reuse
Warning	Can be gamed	Even more gameable with AI

AI-SPACE Metrics:

Metric	Description	Unit	Elite
AI Leverage	% of work performed with AI assistance	%	> 80%
Prompt Reuse Rate	% of prompts templated for reuse	%	> 60%
AI Session Efficiency	Meaningful outcomes per AI session hour	outcomes/hr	> 5
Orchestration Complexity	Multi-step AI workflows created	avg steps	10+
AI-Assisted Feature Delivery	Features delivered per sprint with AI	features/sprint	> 10

AI Leverage (from GitHub's Well-Architected framework) is perhaps the most important new metric. It measures what percentage of development work is performed with AI assistance—the fundamental indicator of AI adoption maturity.

What NOT to measure:

- Lines of code generated by AI (infinite, meaningless)
- Number of AI prompts (easily gamed)
- AI tool usage time (more ≠ better)
- Raw commit frequency (AI can generate infinite commits)
- Code churn (AI refactoring inflates churn without value correlation)

C — Communication and Collaboration (5 metrics)

Aspect	Standard SPACE	AI-SPACE
Focus	Team communication	Human-AI-Human collaboration
Key Metric	PR review time, meetings	AI workflow sharing, feedback loops
New Consideration	N/A	Prompt libraries, AI-assisted documentation

AI-SPACE Metrics:

Metric	Description	Unit	Elite
AI Workflow Sharing Rate	% of AI workflows shared with team	%	> 70%
Prompt Library Contributions	Monthly contributions to shared prompts	count/month	10+
Cross-team Pattern Adoption	AI patterns adopted from other teams	%	> 50%
AI-Enabled Feedback Loop Speed	Time from change to actionable feedback	minutes	< 10
AI-Assisted Documentation Quality	Completeness of AI-maintained docs	%	> 90%

Why it matters: AI changes collaboration patterns. Instead of sharing code snippets, teams share prompts. Instead of code review, teams review AI workflows. The **feedback loop speed** metric captures a key AI advantage—AI enables near-instant feedback on changes through automated review, test generation, and analysis.

E — Efficiency and Flow (5 metrics)

Aspect	Standard SPACE	AI-SPACE
Focus	Uninterrupted work time	AI-augmented flow state
Key Metric	Flow state hours	AI acceleration factor, cycle time
New Consideration	N/A	Context switching to/from AI, time savings

AI-SPACE Metrics:

Metric	Description	Unit	Elite
AI Acceleration Factor	Task completion speed with AI vs without	x	> 5x
Context Switch Frequency	Disruptive switches between human and AI work	/day	< 3
AI-Augmented Flow Hours	Hours per day in AI-augmented flow state	hours/day	> 6
AI-Assisted Cycle Time	Time from work start to production	hours	< 4
Weekly AI Time Savings	Hours saved per week due to AI	hours/week	> 15

Why it matters: Flow state with AI is different from traditional flow. The best AI developers enter a state where AI becomes a seamless extension of their thinking. Poor AI integration creates constant context switching between human reasoning and AI interaction.

The **cycle time** metric (from LinearB) complements lead time—it measures the full span from when work begins to when it reaches production, capturing the entire AI-assisted development cycle.

Anti-Patterns: What Not to Measure

AI-SPACE explicitly identifies metrics that become harmful in AI-assisted development:

Anti-Pattern	Why It Fails
Lines of code	AI can generate infinite LOC; meaningless metric
Number of prompts	Easily gamed; more prompts ≠ better outcomes
AI tool usage time	More time ≠ better; efficiency matters
Raw commit frequency	AI can generate infinite commits
Code churn	AI refactoring inflates churn without value correlation

These metrics were already problematic in traditional development. With AI, they become actively harmful—incentivizing behaviors that reduce rather than improve outcomes.

Framework Mapping: How Metrics Connect

Operations metrics often map to multiple frameworks. A single SLI (Service Level Indicator) can be measured against both standard and AI-modified thresholds.

For example, **Deployment Frequency** maps to:

DORA: Deployment Frequency (DF) — Elite: “Multiple per day”
AI-DORA: Deployment Frequency (DF) — Elite: “10+ per day”
SPACE: Activity — Measuring team output cadence
AI-SPACE: Activity — But shifts focus to AI leverage and orchestration quality

This dual mapping allows organizations to:

Track progress against industry-standard DORA/SPACE benchmarks
Identify AI-specific improvement opportunities
Measure whether AI tooling is delivering expected acceleration

---

Applying the Frameworks by Role

Different roles will use these frameworks differently based on their position in the Product Builder Maturity Model.

For Engineers (L0-L3)

Focus on: AI-SPACE Performance and Efficiency dimensions

Early-stage AI adoption should focus on code quality and acceleration:

Track AI acceptance rate as you learn to work with AI
Monitor catch rate to ensure you’re reviewing effectively, not rubber-stamping
Measure acceleration factor to validate AI is helping
Check defect density to ensure AI code quality is improving

Key question: Is AI making me faster without sacrificing quality?

Warning signs:

Catch rate near zero (rubber-stamping)
Acceleration factor below 2x (AI friction)

- AI quality score declining over time
- Defect density higher than human baseline

For Product-Aware Engineers (L3-L4)

Focus on: AI-DORA metrics, AI Leverage, and Cycle Time

Engineers with product judgment should measure delivery impact:

- Track AI Leverage to measure adoption maturity
- Monitor deployment frequency and lead time against AI-DORA thresholds
- Build reusable prompts and workflows (prompt reuse rate)
- Measure cycle time to validate end-to-end acceleration

Key question: Is AI helping me ship more value to users?

Warning signs:

- High deployment frequency but high failure rate (shipping fast, breaking things)
- Low prompt reuse (reinventing wheels)
- Long cycle times despite AI assistance (non-coding bottlenecks)
- AI Leverage below 60% (underutilizing AI)

For Product Managers Building Prototypes (L2)

Focus on: AI-SPACE Satisfaction and Activity dimensions

PMs using AI to prototype should measure validation speed:

- Track toil reduction (is AI eliminating grunt work?)
- Measure orchestration complexity (can you direct multi-step AI workflows?)
- Monitor AI tool satisfaction (is the tooling enabling or frustrating you?)
- Check feature delivery rate (are you shipping prototypes faster?)

Key question: Can I validate ideas before engineering investment?

Warning signs:

- Low creative work ratio (spending time on AI plumbing, not product thinking)
- Poor session efficiency (AI isn't producing usable outputs)
- Prototypes requiring full engineering rebuild (AI output not production-ready)
- Low AI Leverage (not fully utilizing AI capabilities)

For Product Builders (L5+)

Focus on: All four AI-DORA metrics plus AI-SPACE holistically

Full product builders own end-to-end delivery and should measure everything:

- Track all DORA metrics against AI-DORA thresholds
- Monitor AI Leverage to ensure mature adoption (> 80% for elite)
- Balance speed metrics against quality metrics (defect density, security)
- Track team satisfaction and work-life balance

Key question: Am I delivering complete, reliable products with AI assistance?

Warning signs:

- Optimizing one metric at the expense of others
- Team satisfaction declining despite productivity gains
- Technical debt accumulating invisibly (low code security scores)
- Work-life balance degrading (AI creating pressure, not relief)

Maturity Levels for AI-SPACE

We can map AI-SPACE adoption to maturity levels:

Level	Description	Key Metrics
L0	No AI usage	Standard SPACE applies
L1	Ad-hoc AI	AI satisfaction > 3, some toil reduction, AI leverage < 20%
L2	Structured AI	Prompt reuse > 30%, workflow sharing begins, AI leverage 20-40%
L3	Optimized AI	Acceleration factor > 3x, high quality scores, AI leverage 40-60%
L4	AI-Native	AI leverage > 80%, < 3 context switches/day, cycle time < 4 hours

Most organizations today are between L0 and L2. L3 requires deliberate investment in AI workflow optimization. L4 represents mature AI-native development where AI is the default, not the exception.

Implementation Guidance

Starting with AI-DORA

If you're currently tracking DORA metrics, begin by:

Establish baselines. Measure your current DORA performance.
Add AI-DORA thresholds. Map your performance against AI-DORA elite/high/medium/low.
Identify gaps. Where does AI assistance have the most potential impact?
Instrument AI enablers. Track which AI tools contribute to which metrics.

Starting with AI-SPACE

If you're measuring developer productivity:

Identify anti-patterns. Stop measuring LOC, commits, or other metrics that AI breaks.
Add AI Leverage. This single metric captures overall AI adoption maturity.
Add AI-specific metrics. Start with AI satisfaction, acceleration factor, and defect density.
Expand gradually. Add prompt reuse, workflow sharing, and feedback loop speed as AI adoption matures.
Balance dimensions. Ensure you're measuring across multiple SPACE dimensions, not optimizing one.

Avoiding Common Mistakes

Mistake	Why It's Harmful	Better Approach
Measuring only speed	Ignores quality, satisfaction, sustainability	Balance AI-DORA and AI-SPACE
Comparing AI and human LOC	Meaningless comparison	Focus on outcomes delivered
Mandating AI usage	Creates gaming, reduces satisfaction	Measure value via AI Leverage, not tool usage
Ignoring satisfaction	High velocity with low satisfaction is unsustainable	Include AI satisfaction and work-life balance
Ignoring security	AI can introduce vulnerabilities	Track code security/maintainability explicitly

The Canonical Framework Definitions

The AI-DORA and AI-SPACE frameworks are maintained as part of the ProductBuildersHQ Frameworks:

- **AI-DORA**: `frameworks/ai-dora/ai-dora.json`
- **AI-SPACE**: `frameworks/ai-space/ai-space.json` (25 metrics across 5 dimensions)

Standard DORA and SPACE definitions are maintained in **grokify/prism-maturity**, enabling organizations to compare against industry baselines while measuring AI-specific acceleration.

Conclusion

DORA and SPACE remain essential frameworks for measuring software delivery and developer productivity. But AI development requires modifications to thresholds, new metrics, and fundamentally different interpretations.

AI-DORA provides:

- Recalibrated thresholds for AI-accelerated delivery
- New “elite” targets that reflect AI capabilities
- AI enabler identification for each metric

AI-SPACE provides:

- 25 metrics across 5 dimensions (expanded from original 15)
- AI Leverage as a key adoption indicator
- New metrics for defect density, feedback loops, cycle time, and work-life balance
- Explicit anti-patterns to avoid (LOC, commits, churn)

The shift is not from DORA to AI-DORA or SPACE to AI-SPACE. It is from measuring human-only development to measuring human-AI collaboration. Organizations that

adopt these frameworks early will have clearer visibility into what's working, what's not, and where to invest.

The metrics you choose shape the behaviors you get. Choose metrics that matter for the AI age.

References

Forsgren, Nicole, Jez Humble, and Gene Kim. *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press. 2018.

DORA Research Program. **https://dora.dev**

Forsgren, Nicole, Margaret-Anne Storey, Chandra Maddila, Thomas Zimmermann, Brian Houck, and Jenna Butler. "The SPACE of Developer Productivity." *ACM Queue*. March 2021. **https://queue.acm.org/detail.cfm?id=3454124**

GitHub Well-Architected: Engineering System Metrics. **https://wellarchitected.github.com/library/productivity/recommendations/engineering-system-metrics/**

LinearB. "SPACE Framework: 20 Metrics Defined." **https://linearb.io/blog/space-framework**

Google Cloud. State of DevOps Report. **https://cloud.google.com/devops/state-of-devops**

ProductBuildersHQ. AI-DORA Framework. **https://github.com/ProductBuildersHQ/productbuildershq-frameworks/tree/main/frameworks/ai-dora**

ProductBuildersHQ. AI-SPACE Framework. **https://github.com/ProductBuildersHQ/productbuildershq-frameworks/tree/main/frameworks/ai-space**

This is a living document. Last updated June 2026. Submit feedback at productbuildershq.com.